

Exercise 11: Put a Face On It, Document and Ship

DEV CYCLE	Ship (stakeholder demo, handoff, production readiness)
CC FEATURES	Streamlit UI built by Claude Code, README / HANDOFF / ADR generation, /context, /compact, /cost, generated commits, production gap audit
SPEC TRACES	SPEC §13 Definition of done; §12 traceability reconciled against the real code
TIME	40 min
YOU PRODUCE	ui/app.py (clinician console), README, HANDOFF, an ADR, clean commits, and docs/PRODUCTION-GAP.md

Objective

An API is invisible to the clinicians who have to trust it, and a PoC nobody can pick up is a dead end. You will put a face on DischargeFlow with a small clinician console, run the full Definition-of-Done journey, generate the documentation an engineer needs to take over, commit clean work, and write the honest production gap audit a hospital would ask for before anyone greenlights real patients.

Walk-through

1 Have Claude Code build the clinician console

One file, talking to your running API over httpx, so the UI stays cleanly decoupled from the backend.

TYPE THIS PROMPT INTO CLAUDE CODE

Create ui/app.py: a Streamlit clinician console for DischargeFlow using httpx. Read API_BASE (default http://127.0.0.1:8000) and DISCHARGE_API_KEY from the environment; send the key as X-API-Key on writes. A sidebar field sets "Signed in as" (clinician id). Four tabs. Submit: upload a record JSON file and POST it, then a "Request draft" button. Review queue: list pending drafts from GET /review-queue; for the selected draft show each section, the DB-rendered medication table, citations, reading grade, and safety_flags highlighted in red; Approve and Reject buttons (Reject needs a reason). References: upload .md files and list them. Patient view: pick an approved record and show GET /patient-summary as the patient would see it. Show API errors (403, 409, 422, 503) as friendly messages using the response detail, never a stack trace. Module docstring.

VALIDATE ui/app.py exists, uses httpx (not the ORM directly), reads config from the environment, and disables the submit buttons while a request is in flight.

2 Run both servers

Keep FastAPI running in terminal 1. Open a second terminal, activate the venv there too, and start Streamlit.

TERMINAL 1 (KEEP RUNNING)

```
PS> python -m uvicorn app.main:app --reload
```

TERMINAL 2 · WINDOWS

```
PS> .\.venv\Scripts\Activate.ps1
PS> streamlit run ui/app.py
```

TERMINAL 2 · MACOS / LINUX

```
$ source .venv/bin/activate
$ streamlit run ui/app.py
```

VALIDATE `http://localhost:8501` loads the console. Streamlit calls the API server-side through `httpx`, so there is no browser CORS problem to solve: do not add CORS middleware you do not need.

3 Run the Definition-of-Done journey

This is SPEC §13 item 3, end to end, in the UI. It is also your stakeholder demo.

#	Signed in as	Action	You should see
1	dr.iyer	Submit pneumonia_record.json, request a draft	Draft v1 in the queue, citations from the pneumonia reference
2	dr.iyer	Try to approve your own draft	A friendly 403: "reviewer must not be the submitting clinician"
3	dr.rao	Reject with a specific, actionable reason	Record shows changes_requested; the reason is stored as memory
4	dr.iyer	Request a new draft (DRAFTER_MODE=agent)	v2 visibly applies the feedback
5	dr.rao	Approve v2	Record approved; approving again gives 409
6	anyone	Patient view	Real name, plain language, med table, approval stamp; no MRN or DOB

VALIDATE All six steps behave as listed, and the console shows every guard as a human-readable message. Open `http://127.0.0.1:8000/docs` in another tab: the records you created in the UI are there too. One backend, two surfaces, one source of truth.

SCREENSHOT SLOT *Clinician console: review tab with a safety flag highlighted, and the patient view*

4 Generate the handoff documentation

Documentation-first means a newcomer can run the system and understand its decisions without you. Have Claude write it from the real code, and reconcile the spec while it is at it.

TYPE THIS PROMPT INTO CLAUDE CODE

Write README.md for DischargeFlow from the actual code: what it does, the safety model in five lines, setup on Windows and macOS, .env keys, how to run the API, the UI and the tests, how to connect the MCP server, and the endpoint reference. Then create docs/HANDOFF.md (how it works, the draft sequence, what a deployment engineer must know) and docs/adr/0001-llm-writes-prose-database-writes-doses.md as a short ADR: the decision that doses are fixed-point integers rendered from structured data and never generated by the model, the alternatives considered, and the consequences. Finally compare SPEC §12 with the tests that actually exist and list every mismatch.

VALIDATE README and HANDOFF match the real commands and endpoints, including the ones added since Exercise 5. The ADR states the decision, why (a hallucinated or mis-rounded dose is a patient-safety event), and the trade-off (formatting for display, and the model can only describe medicines, not dose them). Fix every §12 mismatch in the spec or the tests, deliberately.

5 Manage context and cost, then commit clean work

This session is now long. Check it, compact it with instructions so the important context survives, read the bill, and commit in logical pieces.

INSIDE THE CLAUDE CODE SESSION

```
> /context
> /compact Keep: SPEC requirement IDs, architecture decisions, open issues, file layout
> /cost
```

TYPE THIS PROMPT INTO CLAUDE CODE

Review git status and commit the work in logical, conventional commits (feat, test, docs, chore), each message citing the requirement IDs it implements. Do not commit .env, dischargeflow.db or anything in .env. Show me the plan before committing.

VALIDATE /compact keeps a summary of the thread (unlike /clear, which wipes it). /cost shows the session spend. git log --oneline shows a handful of meaningful commits such as feat(review): enforce four-eyes rule (FR-10), and git status is clean. The PreToolUse hook would have blocked any attempt to edit .env; the .gitignore keeps it out of commits.

6 Run the production gap audit

Finally, the honest reckoning. A clinical system has a much higher production bar than an expense tracker. Have Claude assess the gap, classify it and estimate it. This is what you take to stakeholders.

TYPE THIS PROMPT INTO CLAUDE CODE

Audit DischargeFlow against a production bar for a hospital: real authentication, SSO and role-based access; privacy law and consent (for example India's DPDP Act 2023, or HIPAA for US data) and a data-processing agreement and data residency with the LLM vendor; PHI handling end to end, including backups and logs; clinical safety case, hazard log and model evaluation on a golden set of records (hallucination and omission rates); EHR integration (HL7 FHIR); multi-language output; accessibility; audit immutability; rate limiting and cost controls; observability; Postgres, migrations and pooling; secrets management and key rotation; hooks as guardrails not sandboxes; deployment and health checks; incident response. For each: the gap, blocking or deferrable, and a rough effort. Write docs/PRODUCTION-GAP.md.

VALIDATE docs/PRODUCTION-GAP.md lists gaps with blocking/deferrable labels and effort estimates, and the honest total is far larger than the PoC. If it is rosy, push: "what would a hospital's clinical safety officer and a sceptical SRE add to this list?" That gap, made explicit, is the most valuable thing you can hand a stakeholder.

What success looks like

- A clinician can run the full journey in the console, and every safety rule shows up as a clear message.
- README, HANDOFF and an ADR exist and are accurate to the real code; SPEC §12 matches the real tests.
- You managed a long session deliberately with /compact and read the bill with /cost.
- Clean, conventional commits capture the work, and a production gap audit states honestly what shipping would take.

Common pitfalls (Windows and macOS)

- streamlit not found in the second terminal almost always means the venv was not activated there.
- Port already in use: stop the old process, or start Streamlit with --server.port 8502.
- Displaying doses: divide dose_x1000 by 1000 for display only. Never send floats back to the API.
- /compact summarises and keeps the thread; /clear wipes it. Do not /clear right before you need project context for the docs.
- Generated docs drift from code. Treat README, HANDOFF and SPEC §12 as living documents and regenerate them when the code changes.

Stretch goal

Ask Claude to turn docs/PRODUCTION-GAP.md into a one-slide executive summary with a recommendation: greenlight, greenlight-with-conditions, or do-not-ship. Then add a GitHub Actions workflow that runs the offline test suite on every push. You now have a documented, tested, reviewed and ship-assessed clinical PoC.